

Ab tumhara architecture:

- ✓ VPC
- ✓ Public Subnets
- ✓ Private Subnets
- ✓ Internet Gateway
- ✓ ✓ NAT Gateway (very important)
- 📖 **Ab tum ready ho EKS ke liye 🏡 — yahi actual foundation hai**
- ✓ Ab samajh le final network architecture (very important)

Internet



IGW



Public Subnet (ALB, NAT)



NAT Gateway



Private Subnet (EKS nodes, Pods)

📖 Ye **exact same design real companies use karti hain**

✓ Tumne kya achieve kiya (INTERVIEW LEVEL 🔥)

I implemented a production-grade VPC with public and private subnets across multiple availability zones, configured internet gateway for public access, and deployed a NAT Gateway to enable secure outbound internet access for private workloads such as EKS nodes.”**

✓ Ab next — EKS start (MOST IMPORTANT PART ⚡)

Ab tak sab infra foundation tha

📖 Ab hum actual **platform build** start karenge

✓ Step 13 — EKS ke liye prerequisites prepare karna

EKS banaane se pehle hume yeh samajhna zaruri hai 📖

✓ EKS ko kya chahiye?

✓ 1. Networking (✓ DONE)

* VPC ✓

* Subnets ✓

* NAT ✓

✓ 2. IAM roles (next banayenge)

* Cluster role

* Node group role

* IRSA (later)

✓ 3. Subnet tagging (VERY IMPORTANT ⚠)

📖 AWS Load Balancer Controller + EKS ke liye required hota hai

✔ Step 13 — Subnet tagging

Ab hum ****existing VPC module update karenge****

✔ 13.1 — VPC module edit karo

```
bash
cd ~/production-eks-platform/infra/terraform/modules/vpc
nano main.tf
```

✔ 13.2 — Public subnet tags update karo

Search this block:

```
hcl
resource "aws_subnet" "public"
```

Replace tags section with:

```
hcl
tags = {
  Name = "${var.env}-public-${count.index}"
  "kubernetes.io/role/elb" = "1"
}
```

✔ 13.3 — Private subnet tags update karo

Search:

```
hcl
resource "aws_subnet" "private"
```

Replace tags with:

```
hcl
tags = {
  Name = "${var.env}-private-${count.index}"
  "kubernetes.io/role/internal-elb" = "1"
}
```

✔ Yeh kyun important hai? (VERY IMPORTANT)

🔗 EKS aur ALB controller in tags se decide karte hain:

Tag	Meaning
-----	-----
`kubernetes.io/role/elb`	public load balancer yahan bana
`internal-elb`	private load balancer yahan bana

✔ Apply changes

```
bash
cd ~/production-eks-platform/infra/terraform/envs/dev
```

```
terraform plan
terraform apply
```

☞ yeh sirf tagging update karega (no big cost ✔)

✔ Interview GOLD line 🔥

> ****"I properly tagged VPC subnets to integrate with Kubernetes and AWS Load Balancer Controller, ensuring correct placement of public and internal load balancers."****

✔ Simple Hinglish

☞ Yeh tags = EKS ko directions dena
☞ "Public traffic idhar bhejo" / "Private idhar rakho"

✔ Tumhara next step

Ab:

1. Tags add karo ✔
2. terraform apply ✔

✔ Done ho jaye toh bolo:

☞ ****"Step 13 done"*****

🚀 Next step (bahut powerful)

☞ ****EKS cluster create karenge (Terraform se)****
☞ IAM roles
☞ Node group
☞ kubectl connection

☞ Matlab:

🌟 ****tumhara actual Kubernetes platform ready hone wala hai****

Bhai 🔥 sahi pakda — yeh ****subnet tagging wala part sabko confusing lagta hai initially****, lekin ek baar samajh gaya na toh bahut easy hai.

Main tujhe ****real-life analogy + simple Hinglish + EKS logic**** me samjhata hoon 🙌

✔ Problem kya solve kar rahe hain?

Tumne VPC bana liya ✔

Usme public + private subnets bhi bana liye ✔

👉 But EKS ko kaise pata chale:

* kaunsa subnet ****public (internet-facing load balancer)**** ke liye hai?

* kaunsa subnet ****private (internal services)**** ke liye hai?

👉 ****Yahi kaam tagging karti hai ✔****

✔ Simple analogy (best samajh)

Socho tumhare paas 2 rooms hain:

* Room 1 = Public

* Room 2 = Private

👉 Ab koi aadmi (EKS) aata hai aur poochta hai:

> “Main customer ko kahan bithau (public)?”

> “Aur internal team kahan kaam kare (private)?”

👉 Agar label hi nahi hoga toh confuse ho jayega ✖

👉 Agar label laga hai:

* “Public Room” ✔

* “Internal Room” ✔

👉 Kaam easy ho gaya ✔

✔ AWS/EKS me kya hota hai

EKS + AWS Load Balancer Controller yeh tags read karta hai:

✔ Public Subnet Tag

hcl

"kubernetes.io/role/elb" = "1"

☞ iska matlab:

> ****"Yeh subnet public load balancer (ALB) ke liye use karo"****

✓ Private Subnet Tag

```
hcl
"kubernetes.io/role/internal-elb" = "1"
```

☞ iska matlab:

> ****"Yeh subnet internal load balancer ke liye use karo"****

✓ Real flow (VERY IMPORTANT)

Case 1 — Public Load balancer (IMPORTANT 🔥)

Jab tum Kubernetes me likhoge:

```
yaml
type: LoadBalancer
```

☞ AWS automatically ALB banayega

☞ aur woh dekhega:

```
kubernetes.io/role/elb = 1
```

☞ aur ALB ****public subnet me ban jayega ✓****

Case 2 — Internal Load balancer

```
yaml
annotations:
  service.beta.kubernetes.io/aws-load-balancer-internal: "true"
```

☞ AWS dekhega:

```
kubernetes.io/role/internal-elb = 1
```

☞ aur LB private subnet me banega ✓

✔ Agar tagging nahi karein toh?

✗ EKS confused ho jayega

✗ Load balancer wrong subnet me ban sakta hai

✗ Deployment fail bhi ho sakta hai

☞ Interview me yeh bolna powerful hota hai ✔

✔ Simple Hinglish summary

Tag	Purpose
-----	-----
`elb`	Internet-facing load balancer
`internal-elb`	Private/internal load balancer

✔ Tumhare project me kya hoga (real scenario)

* User → ALB (public subnet)

* ALB → pod (private subnet)

☞ Yeh tabhi possible hai jab tagging sahi ho ✔

✔ Interview GOLD answer 🔥 (yaad kar lo)

> **"I tagged public and private subnets appropriately to allow AWS Load Balancer Controller and EKS to automatically place internet-facing and internal load balancers in the correct subnets."**

✔ Aur bhi deep concept (bonus 🔥)

Later hum add karenge:

```
hcl
"kubernetes.io/cluster/<cluster-name>" = "shared"
```

☞ Isse EKS ko pata chalega:

> "Yeh subnets mere cluster ke liye available hain"

✔ Ek line me yaad rakho

☞ ****Subnet tagging = EKS ko direction dena****

✔ Tumhara next step (important 🚀)

Ab hum next phase me ja rahe hain:

👉 ****Step 14 — IAM roles for EKS (cluster + node group)****

👉 (yeh sabse critical security part hai)

✓ bolo:

👉 ****“Step 14 ready”****

“I implemented IAM roles for EKS cluster and worker nodes with least-privilege policies, enabling secure communication between Kubernetes components and AWS services.”

Bhai 🔥 ****perfect timing — EKS samajhne se pehle yeh 3 cheezein clear hona must hai:****

👉 ****Cluster, Control Plane, Nodes****

1. Kubernetes Cluster kya hota hai?

Cluster = pura Kubernetes system (control plane + nodes)**

Cluster = ****ek complete environment jahan tum apni application run karte ho****

Isme hota hai:

Cluster =

Control Plane (brain)

+

Nodes (workers)

Example

Tum bolte ho:

👉 “Mujhe 3 microservices run karni hain”

👉 woh sab cluster ke andar run honggi ✓

✓ Interview answer

> ****“A Kubernetes cluster is a complete system consisting of control plane and worker nodes, responsible for running and managing containerized applications.”****

✓ 2. Control Plane kya hota hai?

Control Plane = Kubernetes ka brain 🧠**

Iska kaam kya hai?

* Decide karta hai kaunsa pod kahan chalega

* Cluster ka state maintain karta hai

* API expose karta hai (kubectl ke liye)

✓ Internal components (sirf understanding ke liye)

* API Server

* Scheduler

* Controller Manager

* etcd (DB)

✓ EKS me kya hota hai?

👉 AWS ****Control Plane manage karta hai****

✓ Tumhe manage nahi karna

✓ Highly available hota hai (multi-AZ)

✓ Simple analogy

☞ Control Plane = **manager / boss**

- * kaam assign karta hai
- * monitoring karta hai
- * system control karta hai

✓ Interview answer

> **“The control plane acts as the brain of the Kubernetes cluster. It manages scheduling, cluster state, and API interactions. In AWS EKS, the control plane is fully managed by AWS and highly available.”**

✓ 3. Nodes kya hote hain?

☞ Simple definition:

> **Nodes = EC2 machines jahan actual application run hoti hai**

Nodes = worker machines
Pods (containers) yahin chalti hain

Example flow
Control Plane → decide karta hai
Node → run karta hai pod

Node ke andar kya hota hai?

- * kubelet (agent)
- * container runtime (Docker/containerd)
- * pods run hote hain

EKS me

- ☞ Nodes = EC2 instances
- ☞ Node group = EC2 ka group

Simple analogy

Component	Role	
-----	-----	
Control Plane	Manager	
Nodes	Employees	

✓ Interview answer

> **“Worker nodes are EC2 instances where Kubernetes pods and containers run. They communicate with the control plane and execute workloads assigned by the scheduler.”**

✓ COMPLETE FLOW (VERY IMPORTANT 🔥)

☞ Real request flow

User → Application request



Control Plane decides



Node selected



Pod runs on Node

✔ Tumhare project me mapping

Component	Tumne kya kiya
-----	-----
Cluster	Abhi create karne wale ho
Control Plane	AWS manage karega
Nodes	Terraform se create karoge
Networking	✔ Already done

✔ Simple 3-line summary (yaad kar lo 🌟)

☞ ****Cluster = system****

☞ ****Control Plane = brain****

☞ ****Nodes = workers****

✔ Interview SUPER GOLD answer 🔥🔥

> ****“In my implementation, I used Amazon EKS where the control plane was fully managed by AWS, and I provisioned worker nodes using EC2 node groups. The control plane handled scheduling and cluster management, while the nodes executed application workloads in pods.”****

✔ Ab next step (IMPORTANT 🚨)

Ab tum ready ho next ke liye:

☞ IAM roles ban chuke ✔

☞ VPC ready ✔

☞ Concept clear ✔

✔ Step 15 — Dev me IAM module call karna

Ab hum:

☞ IAM module ko dev environment me use karenge

☞ phir EKS create karenge

✔ bolo:

☞ ****“Step 15 ready”****

Bilkul bhai — **“cluster ka state”** ka matlab simple words me:

- > **Cluster ki current condition ya current reality.**
- > Matlab Kubernetes ke andar abhi kya-kya chal raha hai, kya desired hai, aur actual me kya status hai.

Kubernetes docs me controllers ka concept yahi hai: controllers cluster ka state watch karte hain aur current state ko desired state ke close laane ki koshish karte hain.
[kubernetes.io](https://kubernetes.io/docs/concepts/architecture/controller/)

Simple example

Maan lo tumne Kubernetes ko bola:

```
yaml
replicas: 3
image: nginx:1.25
```

Iska matlab tumhari **desired state** hai:

- > “Mujhe nginx ke 3 pods chahiye.”

Ab actual me cluster me kya ho sakta hai?

Case 1

3 pods running

✔ Desired state = actual state
Sab sahi hai.

Case 2

2 pods running, 1 pod crash ho gaya

✗ Actual state desired state se different hai.

Ab Kubernetes controller dekhega:

- > “User ne 3 pods maange the, abhi 2 chal rahe hain.”

Phir Kubernetes automatically ek naya pod create karega. Isi ko self-healing bolte hain. Kubernetes Deployment controller desired state ko actual state me convert karta hai controlled way me.
[kubernetes.io](https://kubernetes.io/docs/concepts/workloads/controllers/deployment/?ref=kubernet)

Desired state vs Current state

Term	Meaning
-----	-----

Desired state	Jo tum YAML me define karte ho
Current/Actual state	Jo cluster me abhi real me chal raha hai
Cluster state	Cluster ki complete current information

Cluster state me kya-kya include hota hai?

Cluster state me ye sab information hoti hai:

- * Kitne nodes available hain
- * Kitne pods running/pending/crashed hain
- * Kaunsi deployment me kitne replicas chahiye
- * Kaunsi service kis pod ko traffic bhej rahi hai
- * Kaunsa namespace exist karta hai
- * Kaunsa config/secret apply hai
- * Kaunsa pod kis node par scheduled hai
- * Rollout successful hai ya stuck hai

Real-life analogy

Socho tum restaurant manager ho.

Tum desired state define karte ho:

> “Restaurant me 5 waiters active hone chahiye.”

Actual state:

> “Abhi sirf 3 waiters available hain.”

Manager kya karega?

> “2 aur waiters arrange karo.”

Kubernetes me:

- * Manager = control plane/controller
- * Waiters = pods
- * Desired state = 5 pods
- * Actual state = 3 pods
- * Action = 2 pods create karo

EKS me cluster state kahan hota hai?

Kubernetes me cluster state mainly **etcd** me store hota hai.

But EKS me **etcd** aur control plane AWS manage karta hai, tumhe manually etcd manage nahi karna padta.

Tum `kubectl` se state dekhte ho:

```
bash
kubectl get nodes
kubectl get pods
kubectl get deployments
kubectl describe pod <pod-name>
```

Ye commands cluster se current state/read status lete hain.

Interview answer

> **“Cluster state means the current condition of the Kubernetes cluster, including running pods, nodes, deployments, services, and configuration objects. In Kubernetes, I define the desired state through YAML or Helm, and the control plane continuously compares it with the actual state. If there is any drift, controllers reconcile the cluster to match the desired state.”**

Simple yaad rakho bhai:

> **Desired state = mujhe kya chahiye**
> **Actual state = abhi kya chal raha hai**
> **Control plane = dono ko match karne wala brain**

Ab next me **Step 15 — IAM module ko dev environment me call** karenge.

Tumhari statement ka refined version

> **EKS AWS ki managed Kubernetes service hai. AWS control plane manage karta hai. Cluster ke worker nodes ke upar pods run hote hain, aur pods ke andar containers run hote hain. Usually ek microservice ek container image ke form me pod ke andar run hoti hai. Agar pod Deployment/ReplicaSet ke through manage ho raha hai aur crash ho jaye, toh Kubernetes control plane/controller desired state maintain karne ke liye naya pod create karta hai.**
> Amazon EKS Kubernetes clusters ko AWS par run/manage karne ke liye managed service hai, jahan AWS standard EKS mode me Kubernetes control plane manage karta hai. Kubernetes Deployment desired state define karta hai, aur Deployment controller actual state ko desired state ke paas laata hai.

2 corrections yaad rakho

1. Pods autoscale **automatically by default nahi hote**

Pods tab autoscale hote hain jab hum **HPA — Horizontal Pod Autoscaler** configure karte hain.

Example:

CPU > 70% hua → HPA replicas 2 se 5 kar sakta hai

So correct line:

> **Pods autoscale ho sakte hain, but HPA configure karna padta hai.**

2. Pod crash hone par naya pod tab banta hai jab pod Deployment/ReplicaSet se managed ho
Agar tumne direct standalone pod banaya aur woh crash hua, behavior different ho sakta hai.

Production me hum usually direct pod nahi chalte. Hum use karte hain:

Deployment → ReplicaSet → Pods

Deployment desired replicas maintain karta hai, jaise `replicas: 3`. Agar ek pod crash ho gaya, Deployment/ReplicaSet ensure karega ki 3 pods again running rahein.

Ekdum simple final understanding

EKS Cluster



Control Plane = brain, AWS managed



Worker Nodes = EC2 machines



Pods = smallest deployable unit



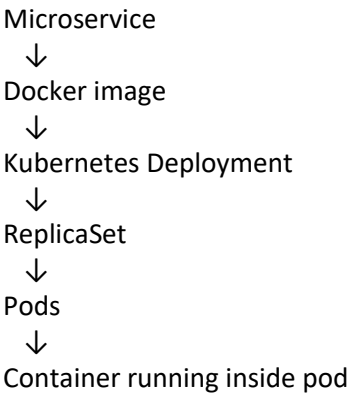
Containers = actual app/microservice

Interview-ready answer

> **"I used Amazon EKS as a managed Kubernetes platform where AWS managed the control plane and I provisioned worker nodes in private subnets. My microservices ran as containers inside Kubernetes pods. I deployed applications using Deployments so Kubernetes maintained the desired replica count. If a pod failed, the controller recreated it automatically, and for scaling I used HPA to increase or decrease pod replicas based on workload metrics."**

Haan, usually har microservice ka apna Docker image hota hai, aur Kubernetes me us microservice ko Deployment ke through pods me run kiya jata hai.

Lekin exact mapping hoti hai:



Kubernetes Deployment desired state define karta hai, jaise kitne replicas chahiye, kaunsi image run karni hai, aur Deployment controller actual state ko desired state ke according maintain karta hai.

Example samjho

Maan lo application me 4 microservices hain:

1. `user-service`
2. `order-service`
3. `payment-service`
4. `notification-service`

Toh normally:

user-service	→ user-service Docker image	→ user-service Deployment	→ user-service pods
order-service	→ order-service Docker image	→ order-service Deployment	→ order-service pods
payment-service	→ payment-service Docker image	→ payment-service Deployment	→ payment-service pods
notification-service	→ notification-service Docker image	→ notification-service Deployment	→ notification-service pods

Kya 4 microservices = exactly 4 pods?

Nahi bhai, exactly 4 pods zaruri nahi.

Agar har microservice ka `replicas: 2` hai, toh:

$4 \text{ microservices} \times 2 \text{ replicas} = 8 \text{ pods}$

Example:

user-service	→ 2 pods
order-service	→ 2 pods
payment-service	→ 2 pods
notification-service	→ 2 pods

Total: 8 pods

Agar traffic jyada aa gaya aur HPA configured hai, toh `payment-service` ke pods 2 se 5 bhi ho sakte hain.

Important point: Pod ke andar kya hota hai?

Usually ek pod ke andar **ek main application container** hota hai.

Example:

Pod: user-service-pod

└─ Container: user-service container

Aur container Docker image se start hota hai:

ECR image: 123456789.dkr.ecr.ap-south-1.amazonaws.com/user-service:v1

Production me flow kaise hota hai?

Developer code likhta hai

↓

CI/CD Docker image build karta hai

↓

Image Amazon ECR me push hoti hai

↓

Kubernetes Deployment image ko refer karta hai

↓

EKS worker node image pull karta hai

↓

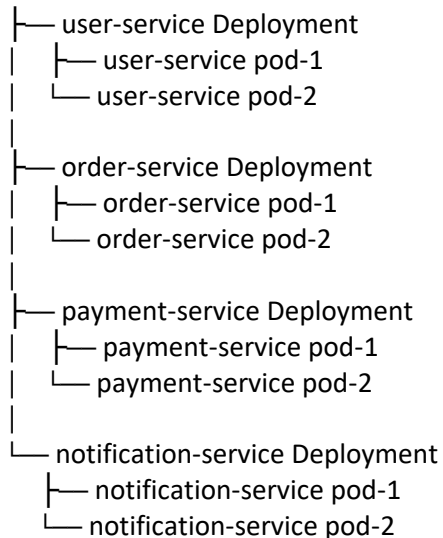
Pod ke andar container start hota hai

↓

Microservice run hoti hai

Simple architecture

EKS Cluster



Interview-ready answer

“In my EKS implementation, each microservice was containerized as a separate Docker image and stored in Amazon ECR. I deployed each microservice using a separate Kubernetes Deployment. Each Deployment managed one or more pod replicas depending on traffic and availability requirements. The pods ran on EKS worker nodes inside private subnets, and Kubernetes maintained the desired replica count through ReplicaSets and controllers.”

Ek line me yaad rakho

Microservice ka code Docker image banta hai, image pod ke container me run hoti hai, aur Deployment decide karta hai kitne pods chahiye.**

Next step me hum **Step 19 apply** kar sakte hain ya agar apply already chal raha/complete hai toh pehle cluster verify karenge.

Haan bhai, **ECS bhi containers run karta hai**, aur practical level par dono ka goal similar hai:

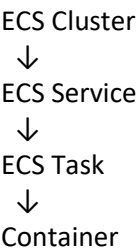
Docker/containerized microservices ko deploy, run, scale, monitor karna.

But **ECS aur EKS ka operating model different hai**.

Simple Difference

ECS = AWS-native container orchestration

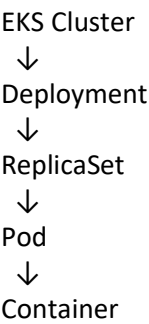
ECS AWS ka apna container orchestration service hai. Isme Kubernetes concepts jaise Pod, Deployment, Service nahi hote. Isme mainly ye concepts hote hain:



ECS simple hota hai, AWS services ke saath tightly integrated hota hai, aur quick setup ke liye achha choice hai. AWS bhi ECS aur EKS dono fully managed container services ke options ke roop me position karta hai.

EKS = Managed Kubernetes

EKS me Kubernetes use hota hai. Isme Kubernetes standard concepts hote hain:



EKS tab strong hota hai jab company Kubernetes ecosystem use karna chahti hai — jaise Helm, ArgoCD, Istio, Kubernetes RBAC, Network Policies, HPA, Karpenter, GitOps etc. EKS AWS managed Kubernetes service hai jahan standard EKS mode me AWS Kubernetes control plane manage karta hai.

ECS vs EKS — Practical Comparison

Point	ECS	EKS
Learning	Easy	Thoda complex
Platform	AWS-native	Kubernetes standard
Concepts	Task, Service, Cluster	Pod, Deployment, Service, Node
GitOps/ArgoCD	Less common	Very common
Helm	Not native	Native Kubernetes ecosystem
Multi-cloud portability	Low	High
Enterprise Kubernetes JD	Limited	Strong match
Operational overhead	Less	More
Cost	No EKS control plane charge	EKS control plane hourly charge
Best for	AWS-only simple container apps	Platform engineering, Kubernetes JDs, complex microservices

Toh hume EKS use karna chahiye ya ECS?

Tumhare case me: ****EKS continue karna chahiye****

Reason simple hai bhai:

Tum jo project bana rahe ho uska goal hai:

- * Kubernetes
- * EKS
- * Helm
- * ArgoCD
- * RBAC
- * Network Policies
- * Ingress Controller
- * Service Mesh basics
- * HPA
- * Karpenter
- * GitOps
- * Production-grade platform

Yeh sab ****EKS/Kubernetes ecosystem**** ke topics hain. Agar hum ECS choose karte, toh Helm, ArgoCD, Kubernetes RBAC, Network Policies, pods, deployments jaise JD topics properly cover nahi hote.

ECS kab choose karna chahiye?

ECS choose karo agar:

- * Company pure AWS-native hai
- * Team ko Kubernetes knowledge nahi hai
- * Fast container deployment chahiye
- * Operational simplicity chahiye
- * App simple web/API services hain
- * Fargate-based serverless containers use karne hain

Interview me bol sakte ho:

For simpler AWS-native container workloads, I consider ECS because it has lower operational complexity and integrates well with AWS services.”**

EKS kab choose karna chahiye?

EKS choose karo agar:

- * Company Kubernetes use karti hai
- * JD me Kubernetes/EKS/Helm/ArgoCD likha hai
- * Multi-service platform banana hai
- * GitOps chahiye
- * Advanced autoscaling chahiye
- * RBAC, Network Policy, Service Mesh, Ingress Controller chahiye
- * Platform engineering role target hai

Interview line:

> ****“For this project, I selected EKS because the requirement was to build a production-grade Kubernetes platform with Helm, ArgoCD GitOps, RBAC, ingress, autoscaling, and platform-level controls.”****

Simple analogy

ECS

> AWS ne ready-made apartment de diya. Tumhe bas apna samaan rakhna hai.

EKS

> Tumhare paas full township planning ka control hai — roads, security, traffic, scaling, policies sab design kar sakte ho.

ECS simpler hai.

EKS powerful hai, but complex bhi hai.

Final decision for our project

✓ **Hum EKS continue karenge.**

Because tumhare interview/JD target ke liye EKS zyada valuable hai.

But interview me ECS ka answer bhi ready rakhenge:

> **“I understand both ECS and EKS. ECS is a simpler AWS-native container orchestration service, while EKS is managed Kubernetes. I used EKS for this platform because the requirement involved Kubernetes-native capabilities like Helm, ArgoCD, RBAC, Network Policies, Ingress Controller, HPA, and GitOps-based delivery.”**

Next actionable step: agar tum ready ho toh bolo **“Step 19 apply ready”**, main tumhe EKS apply ke baad verification commands dunga.

Microservice container ke andar run karegi, aur container pod ke andar run karega.

But pod ka naam container ka naam nahi hota.

Deployment = nginx-test

Image = nginx

Pod = nginx-test-c8697b5cc-494jv

Container = nginx container

Namespace = dev-app

Real project me:

Deployment = user-service

Image = user-service:v1

Pod = user-service-xxxxx

Container = user-service container

Namespace = dev-app

“I deployed a test workload using a Kubernetes Deployment. The Deployment created a ReplicaSet, and the ReplicaSet created a pod. The pod ran one container from the nginx image. In a real microservices setup, each microservice is packaged as a Docker image and runs as a container inside a Kubernetes pod.”

Kubernetes Deployment desired state define karta hai aur pods create/maintain karta hai; tumhare case me Deployment ne ReplicaSet banaya, ReplicaSet ne Pod banaya, aur Pod ke andar nginx container run hua.

Pod ka IP temporary hota hai. Agar pod delete/recreate ho gaya, IP change ho sakta hai. Isliye pod ko directly access karna reliable nahi hota. Kubernetes Service ek stable address/name banata hai jiske through pod ko access kar sakte hain.

Kubernetes docs ke according Service ek method hai jisse cluster ke andar running Pods ko network par expose kiya jata hai, aur Service ek logical set of Pod endpoints ko stable way me accessible banata hai.

Service kya deta hai?

Service deta hai:

Stable virtual IP

Stable DNS name

Load balancing across pods
Pod recreate hone ke baad bhi same access point

Kubernetes DNS Services ke liye DNS records create karta hai, jisse pods Service names se baat kar sakte hain instead of changing IP addresses.

ClusterIP ka matlab :Ye Service sirf cluster ke andar accessible hai, public internet se nahi.

“I created a Kubernetes Service to provide a stable network endpoint for my pods. Since pod IPs are ephemeral and can change when pods restart or scale, I used a Service to route traffic reliably to the healthy pods selected by labels. For internal access, I used a ClusterIP Service.”

Ek Application Run Karne Ka Simple Flow

Socho tumhari application ek shop hai.

EKS Cluster = Mall
Node Group = Mall ke staff/servers ka group
Worker Node = Actual shop floor/server
Namespace = Mall ke andar ek section
Deployment = Shop chalane ka instruction
Pod = Shop ka running unit
Container = Shop ke andar actual machine/app
Service = Shop ka fixed phone number/reception
Port = Kis gate se traffic andar jayega
IP = Temporary address

Tumhare current setup me kya bana?

1. Cluster
dev-eks-cluster

Ye pura Kubernetes environment hai.

2. Worker Nodes
ip-10-0-11-95
ip-10-0-12-23

Ye actual EC2 machines hain jahan pods run hote hain.

3. Namespace
dev-app

Ye application ke liye separate logical area hai.

4. Deployment
nginx-test
Ye Kubernetes ko instruction deta hai:
> “nginx image se app run karo.”

5. Pod
nginx-test-c8697b5cc-494jv
Ye actual running unit hai.

6. Container

nginx
Ye pod ke andar actual app process hai.

7. Service

Tumhare screenshot me:

nginx-test ClusterIP 172.20.219.59 <none> 80/TCP

Ye service pod ke liye stable internal access point hai. Kubernetes Service pods ko stable network endpoint deta hai because pod IP change ho sakta hai.

Sabse simple diagram

User / another pod



Service: nginx-test



Pod: nginx-test-c8697b5cc-494jv



Container: nginx



Application running

Tumhare screenshot ka meaning

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
nginx-test	ClusterIP	172.20.219.59	<none>	80/TCP

Iska matlab:

- * `nginx-test` = Service ka naam
- * `ClusterIP` = sirf cluster ke andar access hoga
- * `172.20.219.59` = service ka stable internal IP
- * `<none>` = public internet se access nahi hai
- * `80/TCP` = service port 80 par listen kar rahi hai

Abhi yaad rakhne wali sirf 5 cheezein

Bas ye 5 yaad rakho, baaki baad me clear hota jayega:

Deployment → Pod banata hai

Pod → Container chalata hai

Container → Application chalata hai

Service → Pod tak traffic pahunchata hai

Node → Pod ko run karta hai

“In my EKS project, I deployed an application using a Kubernetes Deployment. The Deployment created a Pod, the Pod ran the application container, and I exposed the Pod internally using a ClusterIP Service. Since Pod IPs are temporary, I used a Service to provide a stable internal endpoint for accessing the application.”**

Bhai abhi hum dheere challenge.

****Next actionable step:**** ab hum service ko cluster ke andar se test karenge, public ALB abhi nahi.

Correct order simple flow me:

1. EKS Cluster banta hai



2. Worker Nodes join karte hain



3. Namespace banta hai



4. Deployment create hota hai



5. Deployment Pod banata hai



6. Pod ke andar Container run hota hai



7. Service banakar Pod ko stable access diya jata hai

Tumne abhi ClusterIP service banayi hai, iska matlab woh sirf Kubernetes cluster ke andar accessible hai, internet se nahi. Kubernetes Service ka purpose pods ko stable endpoint dena hota hai, kyunki pod IP change ho sakta hai.

Tumhare current project ka flow:

EKS Cluster: dev-eks-cluster



Worker59:80Worker Nodes: 2 Ready nodes



Namespace: dev-app



Deployment: nginx-test



Pod: nginx-test-c8697b5cc-494jv



Container: nginx

Cluster environment hai, Node compute hai, Pod running unit hai, Container app hai, Service access point hai.

Node Group = EC2 worker nodes ka group config

Worker Node = actual EC2 machine

Deployment = kitne pods chahiye

Pod = app run unit

Container = actual microservice process

Service = pod tak stable traffic route

NAT Gateway = private pods/nodes ko outbound internet

“I configured EC2 capacity at the EKS managed node group level using instance type and scaling configuration.

Pod replica count was managed at the Kubernetes Deployment level, and autoscaling boundaries were handled

through HPA. For application access, I used a Kubernetes Service, which does not provide internet access but

gives a stable endpoint to route traffic to pods. Outbound internet from private workloads was provided through

NAT Gateway and private subnet routing.”

Current mental model lock kar lo:

Deployment = desired pods define karta hai

ReplicaSet = pods count maintain karta hai

Pods = actual running units

Container = app process

Node = pod run karne wali EC2 machine

Service = stable access point

Cluster

└─ dev-app namespace

└─ Deployment: nginx-test

└─ Pod 1: 10.0.11.15

└─ Pod 2: 10.0.12.114

└─ Service: nginx-test

└─ routes traffic to both pods

Agar pod stop, delete, crash ya slow ho toh kaise pata chalega?

kubectl get pods → status dekho

kubectl describe pod → reason/events dekho

kubectl logs → app/container logs dekho

kubectl get events → cluster events dekho

kubectl top pods → CPU/memory pressure dekho

CloudWatch/Monitoring → production alerts dekho

“I monitored pod health using kubectl get pods, describe, logs, events, and metrics. If a pod crashed, I checked restart count, events, and previous container logs. I used readiness probes to stop traffic from going to unhealthy or slow pods, and liveness probes to restart containers that were stuck or unhealthy. In production, I used CloudWatch Container Insights and alerts to monitor pod restarts, CPU, memory, node health, and application availability.”

“I validated Kubernetes self-healing by deleting one pod from a Deployment. Since the Deployment desired state was two replicas, Kubernetes automatically recreated a new pod and maintained application availability.”

“I deployed the application using a Kubernetes Deployment. The Deployment maintained the desired state of two replicas through a ReplicaSet, and the ReplicaSet ensured that two pods were always running. When I deleted one pod, Kubernetes automatically created a replacement pod to maintain availability.”

Node group worker nodes ko manage karta hai. Worker nodes pods ko run karte hain. Pods containers ko run karte hain.

“I created an EKS managed node group, which manages the lifecycle and scaling of EC2 worker nodes. The worker nodes are the actual compute machines where Kubernetes pods run. After creating the node group, I validated the cluster using kubectl. I checked the kube-system namespace and verified that critical system pods like aws-node, CoreDNS, and kube-proxy were running, confirming that pod networking, DNS, and service routing were healthy.”

Ab tak humne kya complete kiya?

Phase 1 — AWS Infra Foundation ✓

Terraform structure

S3 backend + DynamoDB locking

VPC

Public/private subnets

NAT Gateway

IAM roles

EKS cluster

Managed node group

Worker nodes Ready

Phase 2 — Kubernetes Basics ✓

Namespace

Deployment

Pod

Container

Service

Endpoints

Scaling replicas

Self-healing

Rolling update

Yeh basic end nahi, yeh production EKS project ka foundation milestone hai.

Abhi project me aur kya-kya karna baaki hai?

Bahut important cheezein baaki hain:

Rollback strategy

Resource requests/limits

Readiness/Liveness probes

HPA autoscaling

ECR + Docker image build

Real application deployment

Helm charts
AWS Load Balancer Controller / Ingress
Public ALB access
RBAC
Network Policies
Secrets / ConfigMap
IRSA
CloudWatch monitoring
ArgoCD GitOps
CI/CD pipeline explanation
FinOps and cleanup/scale-down
Interview-ready architecture story

“I implemented the foundation of a production-grade EKS platform using Terraform, created the EKS cluster and managed node group, deployed a test application, exposed it using a Kubernetes Service, validated self-healing, scaling, and rolling update behavior.”

Haan bhai ✓ ****rollback CLI se kar sakte hain****, lekin ****real production me sirf manually CLI pe depend nahi karte****.

Kubernetes me Deployment rollout revisions maintain karta hai, jisse image/config update ke baad previous revision par rollback possible hota hai.

1. Abhi learning/practical me rollback CLI se karenge

Agar latest rollout me issue aa gaya, toh simple command:

```
bash
kubectl rollout undo deployment/nginx-test -n dev-app
```

Phir status check:

```
bash
kubectl rollout status deployment/nginx-test -n dev-app
```

Phir pods check:

```
bash
kubectl get pods -n dev-app -o wide
```

Aur history:

```
bash
kubectl rollout history deployment/nginx-test -n dev-app
```

2. Specific revision par rollback kaise?

Agar tum revision 1 par rollback karna chahte ho:

```
bash
kubectl rollout undo deployment/nginx-test --to-revision=1 -n dev-app
```

3. Real production me rollback kaise hota hai?

Production me generally 3 tarike hote hain:

Option A — Manual emergency rollback

Production issue aaya, SRE/DevOps engineer emergency me CLI se rollback karta hai:

```
bash
kubectl rollout undo deployment/app-name -n prod
```

Ye fast hota hai, but manual control chahiye.

Option B — CI/CD pipeline rollback

Jenkins/GitHub Actions/CodePipeline me rollback stage hoti hai.

Example flow:

```
Deploy new image
↓
Health check failed
↓
Pipeline rollback job trigger
↓
Previous image/revision deploy
```

Yahan engineer button click karta hai ya pipeline automatic rollback kar sakti hai.

Option C — ArgoCD GitOps rollback

ArgoCD me Git desired state hota hai.

Production me better flow:

```
Bad deployment detected
↓
Git revert / previous manifest restore
↓
ArgoCD sync
↓
Cluster previous stable version par aa jata hai
```

Isme direct cluster me manual changes kam hote hain, Git audit trail maintain hota hai.

Simple difference

kubectl rollback = direct cluster rollback

Pipeline rollback = controlled automation rollback

ArgoCD rollback = Git-based rollback

Interview-ready answer

“For emergency recovery, I used `kubectl rollout undo` to rollback a Kubernetes Deployment to the previous revision. In production, I preferred pipeline-driven or ArgoCD GitOps-based rollback so that every change and rollback was version-controlled, auditable, and repeatable.”**

Step 32 — Ab rollback test karo

Run this:

```
bash
kubectl rollout undo deployment/nginx-test -n dev-app
```

Phir:

```
bash
kubectl rollout status deployment/nginx-test -n dev-app
kubectl get pods -n dev-app -o wide
kubectl rollout history deployment/nginx-test -n dev-app
```

Done ho jaye toh bolo:

☑️ **“Step 32 done”**

“I configured readiness and liveness probes for my Kubernetes workload. Readiness probe ensured that traffic was sent only to pods that were ready to serve requests, while liveness probe allowed Kubernetes to restart unhealthy or stuck containers. Without probes, Kubernetes only knows basic container state, not true application health.”

Metrics Server kya karta hai?

Metrics Server Kubernetes me pods/nodes ka CPU-memory usage collect karta hai aur Kubernetes Metrics API ke through expose karta hai. kubectl top aur HPA CPU/memory autoscaling ke liye isi Metrics API ka use karte hain.

Simple:

Metrics Server nahi hai

↓

kubectl top fail

↓

HPA CPU/memory based autoscaling nahi chalega

Haan bhai ✔️ **nodes aur pods dono ke CPU/memory concept hote hain, but meaning alag hota hai.**

1. Node ka CPU/memory kya hota hai?

Node = actual EC2 machine.**

Toh node ka CPU/memory wahi hota hai jo EC2 instance type provide karta hai.

Example:

Node = t3.small EC2

CPU = instance type ke according vCPU

Memory = instance type ke according RAM

Tumhare `kubectl top nodes` output me ye dikh raha hai:

Node 1 → CPU 25m, Memory 522Mi

Node 2 → CPU 19m, Memory 457Mi

Iska matlab ye nodes apni total EC2 capacity me se itna currently use kar rahe hain. EKS node instance type choose karte time compute, memory, storage, networking aur max pod capacity consider hoti hai.

2. Pod ka CPU/memory kya hota hai?

Pod ka apna physical CPU/memory nahi hota.**

Pod **node ke CPU/memory me se resource use karta hai**.

Simple:

Node ke paas total CPU/memory hoti hai

Pod usme se kuch CPU/memory consume karta hai

Pod/container ke liye hum define kar sakte hain:

requests = minimum resource jo pod ko chahiye

limits = maximum resource jo pod use kar sakta hai

Kubernetes scheduler `requests` dekhkar decide karta hai pod ko kis node par place karna hai, aur kubelet/container runtime `limits` enforce karta hai.

Simple analogy

Socho node ek hotel hai:

Node = Hotel building

CPU/Memory = Hotel ke rooms/resources

Pod = Guest

Request = guest ne minimum kitna room/resources book kiya

Limit = guest maximum kitna use kar sakta hai

Agar node ke paas enough free CPU/memory nahi hai, Kubernetes pod ko us node pe schedule nahi karega.

Real flow

Node: 2 vCPU, 2 GB RAM

↓

Pod A request: 100m CPU, 128Mi memory

Pod B request: 250m CPU, 256Mi memory

Pod C request: 500m CPU, 512Mi memory

Kubernetes scheduler calculate karta hai:

“Kaunsa node in pods ko fit kar sakta hai?”

Important difference

Item	CPU/Memory ka meaning	
-----	-----	
Node	Actual EC2 machine ki total capacity	
Pod	Node ki capacity me se consume/resource allocation	
Container	Actual app process jo CPU/memory use karta hai	
Requests	Minimum guaranteed resource	
Limits	Maximum allowed resource	

`kubectl top nodes` vs `kubectl top pods`

Node level

bash

kubectl top nodes

Batata hai:

Har EC2 worker node ka total CPU/memory usage

Pod level

bash

kubectl top pods -n dev-app

Batata hai:

Har pod kitna CPU/memory consume kar raha hai

Metrics Server CPU/memory metrics Kubernetes Metrics API me expose karta hai, jisse `kubectl top` aur HPA autoscaling kaam karte hain.

Interview-ready answer

“Worker nodes have actual CPU and memory based on the EC2 instance type. Pods do not have separate physical CPU or memory; they consume resources from the worker node. I configured CPU and memory requests and limits at the container level so Kubernetes could schedule pods properly and prevent any workload from over-consuming node resources.”**

Next step

Ab pods ka current CPU/memory usage dekho:

bash

kubectl top pods -n dev-app

Output aa jaye toh bolo:

🔊 **“Step 40 done”**

Iska matlab HPA ke liye required Metrics API available hai. Metrics Server CPU/memory metrics expose karta hai, jise kubectl top aur HPA use karte hain.

HPA Create Karna

Ab hum Horizontal Pod Autoscaler create karenge.

Simple meaning:

HPA automatically pods ka count increase/decrease karta hai based on CPU/memory usage.

Ab HPA bolega:

Minimum pods = 2

Maximum pods = 5

Agar CPU 50% se upar gaya → pods increase karo

“I configured HPA for my Kubernetes Deployment with minimum and maximum replica limits, enabling automatic pod scaling based on CPU utilization collected through Metrics Server.”

“I configured Horizontal Pod Autoscaler for the nginx-test Deployment with minimum 2 replicas and maximum 5 replicas, targeting 50% CPU utilization. Metrics Server provided CPU metrics through the Kubernetes Metrics API, enabling HPA-based autoscaling.”

“I configured HPA for the Kubernetes Deployment using Metrics Server. I set minimum and maximum replica limits and validated that HPA was reading CPU metrics from the Metrics API. For real production workloads, HPA scales pods automatically based on CPU, memory, or custom metrics.”

HPA ne kya sikha diya?

Metrics Server CPU/memory metrics provide karta hai.

HPA Deployment ke pods ko CPU target ke basis par scale kar sakta hai.

Tumne min/max set kiya:

min pods = 2

max pods = 5

target CPU = 50%

Kubernetes HPA Deployment jaise workload ka replica count observed metrics ke basis par automatically adjust karta hai.

Next Concept: Pod Autoscaling vs Node Autoscaling

Abhi humne pods autoscale kiye.

Lekin ek important question aata hai:

Agar pods badh gaye, lekin worker nodes me jagah hi nahi bachi toh?

Tab aata hai:

Cluster Autoscaler / Karpenter

| Scaling Type | Kya scale hota hai? |

| HPA | Pods

| Cluster Autoscaler / Karpenter | Worker nodes / EC2 capacity |

Example:

HPA: pods 2 se 5 karna chahta hai

But nodes me CPU/memory space nahi hai

Then Cluster Autoscaler/Karpenter new EC2 node add karta hai

HPA vs Node Group Scaling:

HPA = pods scale karta hai

Node Group / Cluster Autoscaler / Karpenter = EC2 worker nodes scale karta hai

Example:

HPA dekhta hai CPU high hai



Pods 2 se 5 karne ki try karta hai



Agar nodes me capacity hai → pods schedule ho jayenge



Agar nodes me capacity nahi hai → pods Pending rahenge



Cluster Autoscaler/Karpenter new worker node add karega

Cluster Autoscaler pods ke unschedulable hone aur underutilized nodes ko watch karta hai, aur EC2 Auto Scaling Group ka desired capacity adjust kar sakta hai. Karpenter bhi EKS ke liye node autoscaling solution hai jo workload requirements ke basis par right-sized EC2 capacity provision kar sakta hai.

"I configured pod-level autoscaling using HPA and understood node-level autoscaling separately. HPA increases or decreases pod replicas based on metrics, while Cluster Autoscaler or Karpenter adjusts the worker node capacity when pods cannot be scheduled due to insufficient CPU or memory."

Hum kya microservice bana rahe hain?

Humne jo `support-api` banaya hai, woh ek **backend REST API microservice** hai.

Abhi isme 3 endpoints hain:

GET / → service running message

GET /health → health check for Kubernetes probes

GET /version → deployed version check

Real companies me bhi microservices mostly isi tarah hoti hain:

user-service

order-service

payment-service

notification-service

support-api

ticket-service

inventory-service

Hum abhi **support-api** deploy kar rahe hain as first real microservice.

Real company me flow kaise hota hai?

1. Developer microservice banata hai

Developer apne code repo me API/service banata hai.

Example:

support-api

├─ app.py / Java Spring Boot / Node.js app

├─ requirements.txt / pom.xml / package.json

├─ Dockerfile

└─ tests

Developer ka kaam hota hai:

API code likhna

health endpoint dena

Dockerfile banana

environment variables use karna

unit tests likhna

AWS EKS deployment pattern me microservice ko containerized application ke form me EKS par deploy kiya ja sakta hai, aur image registry ke liye ECR use hota hai. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

2. CI/CD pipeline Docker image banati hai

Real company me developer manually CloudShell me image build nahi karta. Mostly pipeline karti hai:

Git commit



CI pipeline trigger

↓
Build application
↓
Run tests
↓
Build Docker image
↓
Push image to ECR

Amazon ECR Docker container images store karne ke liye managed registry hai, aur Docker image push karne ke liye repository exist honi chahiye, Docker client ECR se authenticate karta hai, phir image push hoti hai.

[\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

3. Platform/DevOps team AWS/EKS me deploy karti hai

Tumhara role yahan aata hai.

Tumhe usually developer ye dega:

Docker image name

image tag

required port

health endpoint

environment variables

CPU/memory requirement

secrets requirement

external/internal access requirement

Example:

Service name: support-api

Image: 628479575897.dkr.ecr.ap-south-1.amazonaws.com/support-api:v1

Port: 8080

Health: /health

Resources:

cpu request: 100m

memory request: 128Mi

Secrets:

DB_HOST

DB_PASSWORD

Phir tum Kubernetes/EKS side par banaoge:

Namespace

Deployment

Service

ConfigMap

Secret

Ingress/ALB

HPA

Probes

Resource limits

Real company architecture flow

Developer Code Repo

↓

CI/CD Pipeline

↓

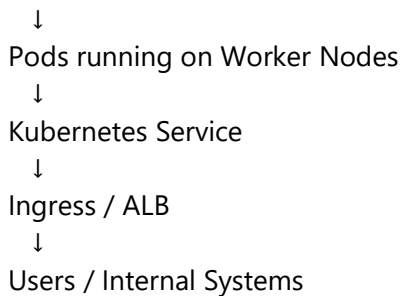
Docker Image Build

↓

Amazon ECR

↓

EKS Deployment



AWS sample EKS deployment flow me Kubernetes deployment, service, namespace aur container image ka use hota hai to run application workloads on EKS. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

Tum interview me kaise bologe?

“In my EKS platform project, developers built individual microservices and packaged them as Docker images. I created the AWS/EKS deployment platform where those images were stored in Amazon ECR and deployed to EKS using Kubernetes Deployments, Services, readiness/liveness probes, resource limits, and HPA. For the first service, I deployed a support-api microservice with health and version endpoints to simulate a real production API deployment flow.”

Ab hum kya karenge?

Ab next real step hoga:

support-api Docker image build

→ ECR repository create

→ image push to ECR

→ Kubernetes deploy from ECR image

Step 51 — ECR Repository Create Karna

Run karo:

```
cd ~/production-eks-platform
```

```
aws ecr create-repository </span>
```

```
--repository-name support-api </span>
```

```
--region ap-south-1 </span>
```

```
--image-scanning-configuration scanOnPush=true
```

Phir verify karo:

```
aws ecr describe-repositories </span>
```

```
--repository-names support-api </span>
```

```
--region ap-south-1
```

Haan bhai, tumhari confusion **bilkul genuine hai**. Real projects me ye confusion tab hoti hai jab “API”, “business logic”, “microservice”, “Docker image”, “ECR”, “EKS deployment” sab ek saath aa jaate hain.

Screenshot me tumhara Docker image successfully build ho gaya hai:

```
support-api v1 132MB
```

Matlab abhi tumhare local CloudShell/Docker environment me `support-api:v1` image ban chuki hai ✓

1. Agar developers ne 10 microservices di toh kya sabki alag Docker image banegi?

Haan, mostly yes ✓

Agar 10 independent microservices hain, toh usually:

user-service → Docker image → ECR repo/image

order-service → Docker image → ECR repo/image

payment-service → Docker image → ECR repo/image

notification-service → Docker image → ECR repo/image

support-api → Docker image → ECR repo/image

Real company me har microservice usually apne codebase ke saath containerized hoti hai aur EKS/ECS jaise container platform par deploy hoti hai. AWS EKS deployment guidance me bhi microservice ko container image ke form me ECR me store karke EKS par deploy karne ka pattern use hota hai. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com), [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

2. Kya har microservice ke liye alag ECR repo hoti hai?

Usually haan, recommended approach yehi hota hai ✓

Example:

ECR Repositories:

- user-service
- order-service
- payment-service
- notification-service
- support-api

Har repo ke andar versions/tags hote hain:

support-api:v1

support-api:v2

support-api:2026-07-01-commit123

ECR Docker images store karne ke liye managed image registry hai, aur image push karne se pehle ECR repository exist honi chahiye. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

3. Developers ke business logic ka Docker image banta hai ya kewal API ka?

Sabse important answer:

Docker image poori runnable application ka banta hai — sirf API endpoint ka nahi.

Developer jo business logic likhta hai, API code likhta hai, dependencies define karta hai — **ye sab Docker image ke andar package hota hai.**

Example real microservice structure:

```
support-api/
├── app.py           → API routes + business logic
├── services/
│   └── ticket_service.py → business logic
├── repositories/
│   └── ticket_repo.py   → DB access logic
├── requirements.txt    → dependencies
└── Dockerfile          → image build instructions
```

Docker image me kya jayega?

Application code ✓

Business logic ✓

API routes/controllers ✓

Dependencies/libraries ✓

Runtime environment ✓

Startup command ✓

Docker image me generally kya nahi jayega?

Database data ✗

Production passwords ✗

AWS credentials ✕

Environment-specific secrets ✕

Secrets/configs Kubernetes Secrets, ConfigMaps, AWS Secrets Manager, Parameter Store, or IRSA ke through handle hote hain.

4. API aur business logic relation kya hai?

Simple example:

Developer API endpoint likhta hai:

POST /tickets

Lekin endpoint ke andar actual kaam hota hai:

validate request

create ticket

save to database

send notification

return response

Ye sab **business logic** hai.

Toh Docker image me sirf /tickets API nahi hoti. Docker image me **poora service application** hota hai.

5. Real company flow exactly kaise hota hai?

Developer code likhta hai

↓

Business logic + API + dependencies commit karta hai

↓

Dockerfile ke through image build hoti hai

↓

CI/CD pipeline test chalati hai

↓

Docker image ECR me push hoti hai

↓

Kubernetes Deployment image ko pull karta hai

↓

Pod ke andar container start hota hai

↓

Container ke andar microservice run hoti hai

Amazon ECR me image push karne ke liye Docker client ko ECR se authenticate karna hota hai, usually `aws`

`ecr get-login-password` ke through Docker login hota hai. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/AmazonECR/latest/userguide/getting-started-cli.html),

[\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/AmazonECR/latest/userguide/getting-started-cli.html)

6. Hum abhi kya bana rahe hain?

Hum abhi first real microservice bana rahe hain:

Microservice name: support-api

Language: Python Flask

Port: 8080

Endpoints:

GET /

GET /health

GET /version

Docker image:

support-api:v1

ECR repo:

support-api

Abhi ye learning ke liye simple hai, but real pattern same hai.

Production me ye support-api aage jaake aisa ho sakta hai:

support-api

- tickets create karega
- DynamoDB/RDS se data lega
- SQS/SNS/EventBridge use karega
- auth validate karega
- logs CloudWatch me bhejega

Interview-ready answer

“In real projects, developers build each microservice as an independent application containing API endpoints, business logic, dependencies, and runtime configuration. I containerize the complete microservice as a Docker image, not just the API endpoint. Each microservice image is versioned and pushed to Amazon ECR, and then deployed to EKS using Kubernetes Deployments, Services, probes, resource limits, and autoscaling configurations.”

Next Step — Step 53: ECR Login + Image Tag + Push

Ab tumhara Docker image local me ready hai:

support-api:v1

Ab isko ECR me push karna hai.

Run these commands:

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
```

```
REGION=ap-south-1
```

```
ECR_URI=$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/support-api
```

```
echo $ECR_URI
```

Phir ECR login:

```
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin
```

```
$ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com
```

```
``
```

Tag image:

```
docker tag support-api:v1 $ECR_URI:v1
```

Push image:

```
docker push $ECR_URI:v1
```

Verify:

```
aws ecr list-images --repository-name support-api --region ap-south-1
```

Haan bhai ✔ screenshot me **Step 53 successfully complete** hai:

```
docker push ... support-api:v1
```

```
aws ecr list-images ...
```

```
imageTag: v1
```

Matlab ab `support-api:v1` Docker image **ECR me push ho chuki hai** ✔

Ab tumhara main confusion clear karte hain.

Short answer

Docker image poori runnable microservice ka banta hai — usme API code, business logic, DB access logic, AWS service call logic, dependencies, runtime sab hota hai.

Lekin **actual database data, SQS queue, SNS topic, EventBridge rule, Step Function workflow infrastructure Docker image ke andar nahi hota.**

Amazon ECR private container images store karta hai, aur images ko EKS/ECS jaise container platforms pull karke run karte hain. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/AmazonECR/latest/userguide/getting-started-cli.html), [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task-repository.html)

Docker image ke andar kya hota hai?

Maan lo developer ne `support-api` microservice diya.
Uska structure real company me kuch aisa ho sakta hai:

`support-api/`

```
├── controllers/
│   └── ticket_controller.py    # API endpoints
├── services/
│   └── ticket_service.py      # business logic
├── repositories/
│   └── ticket_repository.py    # DB insert/read/update/delete logic
├── integrations/
│   ├── sqs_client.py          # SQS message send/receive code
│   ├── sns_client.py          # SNS publish code
│   ├── eventbridge_client.py  # EventBridge event publish code
│   └── stepfunction_client.py  # Step Function start execution code
├── app.py
├── requirements.txt
└── Dockerfile
```

Docker image banate time ye sab code image ke andar package hota hai:

API code ✓

Business logic ✓

DB read/write/delete code ✓

SQS/SNS/EventBridge/Step Function call code ✓

Dependencies ✓

Runtime ✓

Startup command ✓

Docker image ke andar kya nahi hota?

Docker image ke andar ye cheezein generally nahi hoti:

Actual RDS/DynamoDB data ✗

Actual SQS queue ✗

Actual SNS topic ✗

Actual EventBridge rule ✗

Actual Step Function state machine ✗

Production passwords/secrets ✗

AWS access keys ✗

Ye sab infrastructure/configuration side par banta hai:

Terraform / CloudFormation / AWS Console / CI-CD pipeline

Aur app runtime par in resources ko call karta hai.

Example: DB insert/read/delete ka logic

Agar developer ne API banayi:

POST `/tickets`

GET `/tickets/{id}`

DELETE `/tickets/{id}`

Toh Docker image ke andar code hoga:

POST `/tickets` API endpoint

↓

`ticket_service.create_ticket()`

↓

`ticket_repository.insert_to_db()`

↓

RDS/DynamoDB me data save

Yahan `insert_to_db()` ka **code Docker image me hota hai**, but **database Docker image ke andar nahi hota**.

Database AWS me separately hota hai:

RDS / DynamoDB

Example: SQS/SNS/EventBridge/Step Functions logic

Agar business flow hai:

Ticket create hua

↓

EventBridge event publish karo

↓

SQS queue me message bhejo

↓

SNS notification bhejo

↓

Step Function workflow start karo

Toh Docker image ke andar ye code ho sakta hai:

```
eventbridge.put_events(...)
```

```
sqs.send_message(...)
```

```
sns.publish(...)
```

```
stepfunctions.start_execution(...)
```

Lekin actual AWS resources separately create honge:

EventBridge Bus / Rule

SQS Queue

SNS Topic

Step Function State Machine

IAM Role / Policy

Toh hum basically kiska Docker image banate hain?

Bhai simple line yaad rakho:

Hum poori microservice application ka Docker image banate hain — jisme API layer + business logic + integration logic + dependencies sab hota hai.

Sirf API ka image nahi banta.

Sirf business logic ka image nahi banta.

Complete runnable service ka image banta hai.

Real company me responsibility split

Developer ka kaam

Developer banata hai:

API endpoints

Business logic

DB access logic

AWS SDK integration code

Unit tests

Dockerfile

DevOps / Platform / Cloud team ka kaam

Tum deploy karte ho:
ECR repo
EKS Deployment
Kubernetes Service
Ingress / ALB
ConfigMap
Secrets
IAM / IRSA
RDS / DynamoDB / SQS / SNS / Step Functions infra
Monitoring
Autoscaling
CI/CD pipeline

Real production flow

Developer code commit karta hai
↓
Pipeline tests run karti hai
↓
Docker image build hoti hai
↓
Image ECR me push hoti hai
↓
EKS Deployment image pull karta hai
↓
Pod ke andar container start hota hai
↓
Container ke andar full microservice run hoti hai
↓
Microservice DB/SQS/SNS/EventBridge/Step Functions ko call karti hai

Tumhare current `support-api:v1` me kya hai?

Abhi humare image me simple code hai:
GET /
GET /health
GET /version
Abhi DB/SQS/SNS/EventBridge/Step Functions logic nahi hai.
Lekin later hum isi `support-api` me add kar sakte hain:
POST /tickets
GET /tickets
DynamoDB insert/read
SQS message send
SNS notification
EventBridge event publish
Step Function workflow trigger
Tab image dobara build hogi:
`support-api:v2`
Phir ECR me push hogi:
`support-api:v2`
Phir EKS deployment update hoga:
`image: support-api:v2`

Interview-ready answer

“In my project, the Docker image represented the complete runnable microservice, not just an API endpoint. It included API routes, business logic, database access code, AWS SDK integration logic, dependencies, and runtime configuration. External resources like RDS, DynamoDB, SQS, SNS, EventBridge, and Step Functions were provisioned separately using infrastructure automation, and the microservice interacted with them at runtime using IAM-secured access.”

Perfect bhai ✔ **Step 54 — support-api:v1 ko EKS me deploy karte hain.**

Ab hum ECR me pushed image ko EKS worker nodes par run karenge. EKS pod image ko ECR se pull karega, aur container ke andar support-api microservice run hogi. ECR private registry me image store karta hai, aur Kubernetes/EKS deployment image ko pull karke pod/container run karta hai. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com), [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

Step 54 — Support API Deployment Manifest Banao

CloudShell me run karo:

```
cd ~/production-eks-platform
```

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
```

```
REGION=ap-south-1
```

```
ECR_URI=$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/support-api
```

```
echo $ECR_URI
```

Ab deployment YAML file banao:

```
mkdir -p k8s/support-api
```

```
cat > k8s/support-api/deployment.yaml <<EOF
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: support-api
```

```
  namespace: dev-app
```

```
  labels:
```

```
    app: support-api
```

```
spec:
```

```
  replicas: 2
```

```
  selector:
```

```
    matchLabels:
```

```
      app: support-api
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: support-api
```

```
    spec:
```

```
      containers:
```

```
        - name: support-api
```

```
          image: ${ECR_URI}:v1
```

```
          ports:
```

```
            - containerPort: 8080
```

```
          env:
```

```
            - name: APP_NAME
```

```
              value: "support-api"
```

```
            - name: APP_VERSION
```

```
              value: "v1"
```

```
          resources:
```

```
requests:
  cpu: "100m"
  memory: "128Mi"
limits:
  cpu: "250m"
  memory: "256Mi"
readinessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 20
```

EOF

Apply karo

```
kubectl apply -f k8s/support-api/deployment.yaml
Phir rollout check karo:
kubectl rollout status deployment/support-api -n dev-app
Phir pods check karo:
kubectl get pods -n dev-app -l app=support-api -o wide
Expected:
support-api-xxxxx 1/1 Running
support-api-yyyyy 1/1 Running
```

Is step me kya hua?

Humne EKS me real microservice deploy ki:
ECR image: support-api:v1
↓
Kubernetes Deployment: support-api
↓
2 Pods
↓
Each Pod has support-api container
↓
Container runs Flask API on port 8080

Interview line

"I deployed the support-api microservice to Amazon EKS using the Docker image stored in Amazon ECR. I configured replicas, resource requests and limits, environment variables, and readiness/liveness probes to follow production workload patterns."

Bilkul bhai, Step 54 ko simple real-company style me samjho. Is step ka goal tha:

ECR me jo support-api:v1 image push hui hai, usko EKS cluster ke andar pod/container ke form me run karna.

1. CloudShell me humne kya run kiya?

Pehle humne image URI banaya:

```
cd ~/production-eks-platform
```

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
```

```
REGION=ap-south-1
ECR_URI=$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/support-api
```

```
echo $ECR_URI
```

Iska output kuch aisa hoga:

```
628479575897.dkr.ecr.ap-south-1.amazonaws.com/support-api
```

Ye wahi ECR image path hai jahan tumne image push ki:

```
support-api:v1
```

Matlab final image:

```
628479575897.dkr.ecr.ap-south-1.amazonaws.com/support-api:v1
```

2. YAML file kyon banayi?

Kubernetes me production-style deployment ke liye hum command line se manually app run nahi karte. Hum YAML manifest banate hain jisme desired state likhte hain.

YAML ka matlab:

"Kubernetes, mujhe ye application is configuration ke saath run karni hai."

Kubernetes Deployment desired state define karta hai aur Pods/ReplicaSet ke through application workload manage karta hai. Deployment rolling updates aur rollback support karta hai. [\[kubernetes.io\]](https://kubernetes.io)

3. YAML file me kya details daali?

Humne ye details daali:

A. Deployment name

```
name: support-api
```

```
namespace: dev-app
```

Matlab app `dev-app` namespace me deploy hogi.

B. Replicas

```
replicas: 2
```

Iska matlab:

1 microservice ke 2 pods run honge.

So haan bhai ✓

Humne 1 microservice `support-api` ko 2 pods me deploy karne ka instruction diya.

```
support-api microservice
```

```
├── support-api pod 1
```

```
└── support-api pod 2
```

C. Container image

```
image: ${ECR_URI}:v1
```

Matlab pod ke andar container ECR se image pull karega:

```
support-api:v1
```

ECR private container images store karta hai, jise EKS/ECS jaise container platforms pull karke run karte hain.

[\[docs.aws.amazon.com\]](https://docs.aws.amazon.com), [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

D. Container port

```
containerPort: 8080
```

Ye batata hai ki container ke andar Flask app port 8080 par listen kar rahi hai.

Important:

`containerPort: 8080` ka matlab public URL nahi hai. Ye sirf container ke andar application ka internal listening port hai.

E. Environment variables

env:

```
- name: APP_NAME
```

```
  value: "support-api"
```

```
- name: APP_VERSION
```

value: "v1"

Ye app ke runtime configuration values hain.

Real production me yahan aise values hoti hain:

APP_ENV=prod

DB_HOST=...

LOG_LEVEL=info

FEATURE_FLAG=true

Sensitive values direct YAML me nahi daalte; unke liye Secrets Manager / Kubernetes Secret use karte hain.

F. Resources

resources:

requests:

cpu: "100m"

memory: "128Mi"

limits:

cpu: "250m"

memory: "256Mi"

Iska matlab:

Request = minimum CPU/memory expectation

Limit = maximum allowed CPU/memory

Ye production me zaruri hai taaki ek pod poore node ka CPU/memory consume na kar le.

G. Readiness/Liveness probes

readinessProbe:

httpGet:

path: /health

port: 8080

livenessProbe:

httpGet:

path: /health

port: 8080

Iska matlab Kubernetes /health endpoint call karke check karega:

Pod traffic lene ke liye ready hai ya nahi

Pod/container unhealthy/stuck toh nahi

4. kubectl apply kya karti hai?

Command:

kubectl apply -f k8s/support-api/deployment.yaml

Iska matlab:

Kubernetes cluster me is YAML file ke according resources create/update karo.

Agar Deployment pehle se nahi hai, create karega.

Agar Deployment already hai, update karega.

Simple:

YAML file = desired state

kubectl apply = desired state ko cluster me apply karo

5. kubectl rollout status kya karti hai?

Command:

kubectl rollout status deployment/support-api -n dev-app

Iska matlab:

Deployment successfully rollout hua ya nahi, ye watch/check karo.

kubectl rollout status latest rollout ka status watch karta hai jab tak rollout complete na ho jaye.

[\[kubernetes.io\]](https://kubernetes.io)

Ye check karta hai:

new pods create hue ya nahi

Pods ready hue ya nahi
old Pods replace hue ya nahi
rollout stuck toh nahi
Agar sab sahi hai:
Deployment "support-api" successfully rolled out

6. Kya humne 1 microservice ko 2 Pods me deploy kiya?

Haan ✔

YAML me:

replicas: 2

Iska matlab:

1 microservice = support-api

2 Pod replicas = high availability / load distribution

Flow:

Deployment: support-api

↓

ReplicaSet

↓

Pod 1: support-api container

Pod 2: support-api container

Dono Pods same image run karenge:

support-api:v1

7. Port 8080 diya hai — production me URL kaise denge?

Bahut important question 🔥

Abhi 8080 sirf container ke andar app ka port hai.

Production access ke liye layers hoti hain:

User

↓

Route 53 DNS

↓

Application Load Balancer

↓

Ingress

↓

Kubernetes Service

↓

Pods

↓

Container port 8080

Internal Kubernetes Service example

Hum later Service banayenge:

port: 80

targetPort: 8080

Iska matlab:

Service port 80 par request lega

Pod ke container port 8080 par forward karega

Production URL example

Production me user ko aisa URL milega:

<https://support-api.company.com>

``

Behind the scenes:

support-api.company.com

↓ Route 53

ALB

↓

Ingress rule

↓
Kubernetes Service: support-api
↓
Pod: support-api
↓
Container: port 8080
So user ko port 8080 nahi diya jata. User ko clean DNS URL diya jata hai:
<https://support-api.company.com>
Port mapping internal hoti hai.

Real-company explanation
Developer bolta hai:
Meri support-api app port 8080 par chalti hai.
Health endpoint /health hai.
Docker image ECR me v1 tag ke saath available hai.
Platform/DevOps engineer ka kaam:
Deployment create karna
Service create karna
Ingress/ALB configure karna
DNS Route 53 me map karna
TLS certificate attach karna
Monitoring/autoscaling/probes configure karna

Interview-ready answer

"I deployed the support-api microservice to EKS using a Kubernetes Deployment manifest. The manifest defined the ECR image, two pod replicas, container port 8080, environment variables, CPU/memory requests and limits, and readiness/liveness probes. I used `kubectl apply` to apply the desired state to the cluster and `kubectl rollout status` to verify that the deployment completed successfully. In production, I expose the application through a Kubernetes Service, AWS Load Balancer Controller Ingress, ALB, Route 53 DNS, and HTTPS certificate rather than exposing container port 8080 directly."

Bilkul bhai ✔ **Step 55 done** — screenshot me Service successfully create ho gayi hai.

Tumhare output me ye dono important commands hain:

```
kubectl get svc support-api -n dev-app  
kubectl get endpoints support-api -n dev-app
```

1. `kubectl get svc support-api -n dev-app` kya dikha raha hai?

Output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
support-api	ClusterIP	172.20.57.109	<none>	80/TCP	53s

Iska matlab:

support-api

Ye Service ka naam hai.

TYPE = ClusterIP

Iska matlab ye Service **sirf Kubernetes cluster ke andar accessible hai**, public internet se nahi.

CLUSTER-IP = 172.20.57.109

Ye Service ka stable internal IP hai.

Pods ke IP change ho sakte hain, lekin Service ka internal IP stable access point deta hai. Kubernetes Service ka purpose pods ke group ko stable network endpoint ke through expose karna hota hai.

EXTERNAL-IP = <none>

Iska matlab abhi ye public internet par expose nahi hai.

Public URL ke liye later hum use karenge:

Ingress → AWS Load Balancer Controller → ALB → Route 53

PORT(S) = 80/TCP

Service port 80 par listen kar rahi hai.

Lekin yaad rakho:

port: 80
targetPort: 8080
Matlab:
Service port 80 par traffic lega
Aur pod ke container port 8080 par forward karega
Flow:
support-api Service :80
↓
support-api Pod :8080
↓
Flask app running

2. `kubectl get endpoints support-api -n dev-app` kya dikha raha hai?

Output:

NAME	ENDPOINTS	AGE
support-api	10.0.11.19:8080,10.0.12.238:8080	66s

Iska matlab:
Service ke peeche currently 2 healthy pod endpoints registered hain:
support-api Service
├── Pod 1: 10.0.11.19:8080
└── Pod 2: 10.0.12.238:8080
Ye dono wahi 2 `support-api` pods hain jo tumne deploy kiye:
`support-api-76d5445dcc-np7qr` IP 10.0.11.19
`support-api-76d5445dcc-w7qlg` IP 10.0.12.238
Kubernetes Service selector ke basis par matching pods ko endpoints ke form me discover karta hai. Service clients ko stable DNS/IP deta hai, aur backend pods change hone par endpoints update hote rehte hain.

Simple diagram
Internal caller / another pod
↓
Service: support-api
ClusterIP: 172.20.57.109
Port: 80
↓
Endpoints:
10.0.11.19:8080
10.0.12.238:8080
↓
support-api containers
Flask app on port 8080

Warning ka kya matlab hai?
Output me warning hai:
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
Iska matlab command abhi kaam kar rahi hai, but newer Kubernetes versions me `EndpointSlice` recommended hai.
Abhi learning ke liye koi issue nahi ✓
Later production me hum check kar sakte hain:
`kubectl get endpointslice -n dev-app`

Interview-ready answer

"I created a ClusterIP Service for the support-api microservice. The Service exposed port 80 internally and forwarded traffic to container port 8080 on the support-api pods. I verified the Service and endpoints, confirming that both pod replicas were registered behind the Service and ready to receive traffic."

Next Step — Step 56: Service ko Cluster ke andar se Test Karna

Ab hum verify karenge ki Service actually API ko route kar rahi hai ya nahi.

Run karo:

```
kubectl run curl-support-api </span>
--image=curlimages/curl </span>
--rm -it </span>
--restart=Never </span>
-n dev-app </span>
-- curl http://support-api/health
```

Expected output:

```
{"app":"support-api","status":"healthy","version":"v1"}
```

Bhai no issue ✔ CloudShell disconnect ke baad pehle context/path set kar lete hain.

1. CloudShell me kaunsa path use karna hai?

Abhi hum **Kubernetes manifests / app deployment** wale kaam kar rahe hain, isliye root project folder me jao:

```
cd ~/production-eks-platform
```

```
pwd
```

```
`
```

Expected:

```
/home/cloudshell-user/production-eks-platform
```

Agar folder nahi mile toh:

```
find ~ -type d -name "production-eks-platform" 2>/dev/null
```

2. EKS context refresh karo

CloudShell disconnect ke baad safe hai kubeconfig refresh karna:

```
aws eks update-kubeconfig --region ap-south-1 --name dev-eks-cluster
```

Phir check:

```
kubectl get nodes
```

```
kubectl get pods -n dev-app
```

Expected:

```
nodes Ready
```

```
support-api pods Running
```

3. AWS Load Balancer Controller kya hota hai?

Simple bhai:

AWS Load Balancer Controller EKS cluster ke andar ek controller hota hai jo Kubernetes Ingress ya LoadBalancer Service ko dekhkar AWS me ALB/NLB create karta hai.

Matlab tum Kubernetes me ek `Ingress` YAML likhte ho, aur AWS Load Balancer Controller automatically AWS me **Application Load Balancer** create karta hai. AWS docs ke according controller Kubernetes Ingress create hone par AWS ALB create karta hai, aur Service type LoadBalancer par NLB create kar sakta hai.

[\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

Abhi tumhara `support-api` sirf cluster ke andar accessible hai:

ClusterIP Service

Public user ke liye hume chahiye:

User

↓

Route 53 DNS

↓

AWS ALB

↓

Ingress

↓

Kubernetes Service

↓

support-api Pods

4. Ingress kya hota hai?

Ingress Kubernetes ka HTTP/HTTPS routing rule hota hai.

Ingress me hum define karte hain:

Agar request /support ya / aaye

toh support-api Service ko bhejo

Example real production:

<https://support-api.company.com>

↓

Ingress rule

↓

support-api Service

↓

support-api Pods

Ingress khud ALB nahi hai.

Ingress ek Kubernetes object/rule hai.

AWS Load Balancer Controller us Ingress ko read karke AWS ALB create karta hai. AWS documentation me

AWS Load Balancer Controller ko internet traffic route karne ke liye use kiya jata hai, jahan controller

Kubernetes Ingress/Service resources ko watch karke AWS load balancers provision karta hai.

[\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

5. Helm ka role yahan kya hai?

AWS Load Balancer Controller ko cluster me install karne ke liye hum **Helm** use karenge.

Helm simple:

Kubernetes ka package manager.

Jaise `apt install` Linux packages ke liye hota hai, waise Helm Kubernetes components install karta hai.

AWS docs me AWS Load Balancer Controller ko Helm se install karne ka process diya gaya hai.

[\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

6. Ab next command kahan chalani hai?

Ye command **kisi bhi path se chal sakti hai**, but clean rehne ke liye project root se chalao:

`cd ~/production-eks-platform`

Step 59 — OIDC Check

AWS Load Balancer Controller ko AWS APIs call karni hoti hain. Uske liye later hum IRSA role banayenge.

Pehle cluster ka OIDC issuer check karte hain:

```
aws eks describe-cluster </span>
```

```
--name dev-eks-cluster </span>
```

```
--region ap-south-1 </span>
```

```
--query "cluster.identity.oidc.issuer" </span>
```

```
--output text
```

Expected output:

<https://oidc.eks.ap-south-1.amazonaws.com/id/xxxxxxx>

Bhai ✓ screenshot me OIDC issuer URL aa gaya:

<https://oidc.eks.ap-south-1.amazonaws.com/id/37D303D973C28E30E05FBA23B1EC8840>

``

Matlab EKS cluster ke paas OIDC issuer available hai. Ab samjho **OIDC** aur **IRSA role** kya hote hain.

1. OIDC kya hota hai?

OIDC = OpenID Connect

Simple Hinglish me:

OIDC ek trusted identity bridge hai jo Kubernetes Service Account ko AWS IAM ke saath connect karne me help karta hai.

EKS cluster ka apna OIDC issuer URL hota hai. AWS IAM ko batana padta hai:

“Is EKS cluster ke service accounts par trust kar sakte ho.”

AWS docs ke according EKS cluster ke OIDC issuer URL ke liye IAM OIDC provider exist hona chahiye, tabhi IAM roles for service accounts use ho sakte hain. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

2. IRSA role kya hota hai?

IRSA = IAM Roles for Service Accounts

Simple:

IRSA ka matlab hai Kubernetes pod ko AWS IAM role dena, bina access key/secret key container me daale.

AWS docs ke according IRSA me IAM role ko Kubernetes service account ke saath associate kiya jaata hai, aur jo pods us service account ko use karte hain woh AWS services ko IAM permissions ke saath access kar sakte hain. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com), [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

3. Hume AWS Load Balancer Controller ke liye IRSA kyon chahiye?

AWS Load Balancer Controller EKS cluster ke andar chalega, lekin usko AWS me ALB create karna padega.

Usko AWS API calls karni hongi:

Create ALB

Create Target Group

Register Pod targets

Create Listener

Manage Security Group rules

Agar controller ke pod ko AWS permissions nahi milengi, toh woh ALB create nahi kar paayega.

So hum kya karenge:

AWS Load Balancer Controller Pod

↓ uses

Kubernetes Service Account

↓ mapped through IRSA

IAM Role with ALB permissions

↓ can create/manage

AWS ALB resources

AWS Load Balancer Controller Kubernetes Ingress/Service resources ko watch karta hai and AWS ALB/NLB provision karta hai. [\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

4. Without IRSA kya problem hoti?

Old/bad way:

Worker node role ko broad permissions de do

Problem:

Node par jitne pods hain sab indirectly broad AWS permissions access kar sakte hain

Better way with IRSA:

Sirf aws-load-balancer-controller service account ko ALB permissions do

AWS docs ke according IRSA least privilege enable karta hai because IAM permissions service account level par scope hoti hain, aur sirf wahi pods access karte hain jo us service account ko use karte hain.

[\[docs.aws.amazon.com\]](https://docs.aws.amazon.com)

Simple analogy

Socho office me:

OIDC = company ID verification system

Service Account = employee ID card

IAM Role = permission badge

IRSA = employee ID card ko permission badge se link karna

AWS Load Balancer Controller ko ALB create karna hai, toh usko special badge chahiye.

Interview-ready answer

“I used IRSA to give AWS permissions to Kubernetes workloads securely. Instead of storing AWS keys inside containers or giving broad permissions to worker nodes, I associated an IAM role with a Kubernetes service account using the EKS OIDC provider. This allowed components like AWS Load Balancer Controller to create and manage ALBs with least-privilege access.”

Next Step — Step 60: Check IAM OIDC Provider Exists

Ab hum check karenge ki tumhare AWS account me is EKS cluster ke OIDC issuer ke liye IAM OIDC provider already created hai ya nahi.

Run this from:

```
cd ~/production-eks-platform
```

Then run:

```
OIDC_ID=$(aws eks describe-cluster </span>
--name dev-eks-cluster </span>
--region ap-south-1 </span>
--query "cluster.identity.oidc.issuer" </span>
--output text | cut -d '/' -f 5)
```

```
echo $OIDC_ID
```

```
aws iam list-open-id-connect-providers | grep $OIDC_ID || echo "OIDC provider not found"
```

Agar output me OIDC provider ARN aa jaye, bolo:

👉 **“Step 60 done - OIDC provider exists”**

Agar output aaye:

OIDC provider not found

Toh bolo:

👉 **“OIDC provider not found”**

Bhai sach bolu — **tumhari confusion 100% valid hai**. Kubernetes/EKS me first time ye sab dekhkar kisi ko bhi lagega:

“Ek API chalani thi, yahan Controller, Ingress, Service, Target Group, ALB, Pods, Ports, IPs — itna sab kyon?”

But good news ye hai: **tumne jo screenshot bheja hai usme full public flow working hai** ✓

Tumhari API public ALB URL se response de rahi hai:

```
{"app":"support-api","status":"healthy","version":"v1"}
```

Aur `/`, `/health`, `/version` teeno endpoints working hain ✓

Sabse simple mental model yaad rakho

Bas ye flow yaad rakho:

User

↓

ALB

↓

Ingress

↓

Service

↓

Pods

↓

Container

↓

Application

Ab har component ko ek line me samjho.

1. ALB kya hai?

ALB AWS ka public load balancer hai.

Ye internet se request receive karta hai:

<http://k8s-devapp-supporta-xxxx.ap-south-1.elb.amazonaws.com>

Production me yahi ALB Route 53 domain ke peeche hoga:

<https://support-api.company.com>

AWS Load Balancer Controller Kubernetes Ingress ko dekhkar AWS me ALB create/manage karta hai.

[oneuptime.com]

2. AWS Load Balancer Controller kya hai?

Controller ek automation worker hai.

Tumne Kubernetes me Ingress YAML banayi. Controller ne usko dekha aur AWS me ALB create kar diya.

Simple:

Ingress YAML likhi

↓

Controller ne dekha

↓

AWS me ALB bana diya

Controller khud traffic route nahi karta. Controller sirf AWS ALB resources create/manage karta hai.

[oneuptime.com]

3. Ingress kya hai?

Ingress HTTP routing rule hai.

Tumhare Ingress me rule hai:

Path /

↓

support-api Service

Matlab:

ALB par jo bhi request / path par aaye,

support-api service ko bhejo.

Ingress khud application nahi hai.

Ingress ek rule book hai.

4. Service kya hai?

Service pods ka stable internal address hai.

Tumhari service:

support-api:80

Iska kaam:

Service port 80 par request lo

↓

Pod ke port 8080 par bhejo

Kubernetes Service pods ko stable network endpoint deti hai, kyunki pod IP change ho sakta hai.

5. Target Group kya hai?

Target Group ALB ka backend list hai.

Tumhare case me ALB ke target group me pod IPs registered hain:

10.0.12.238:8080

10.0.11.27:8080

Screenshot me bhi dikha:

Backends: support-api:80 (10.0.12.238:8080,10.0.11.27:8080)

Simple:

ALB target group = ALB ko pata hai traffic kin pods ko bhejna hai

Tum direct target group nahi banate. AWS Load Balancer Controller Ingress/Service ke basis par target group create/manage karta hai. [\[oneuptime.com\]](https://oneuptime.com)

6. Pod kya hai?

Pod actual running unit hai.

Tumhari support-api microservice ke 2 pods chal rahe hain:

support-api pod 1

support-api pod 2

Pod ke andar container run hota hai.

7. Container kya hai?

Container ke andar tumhari application chalti hai.

Tumhari application:

support-api Flask app

Port:

8080

Flow:

Pod

└─ Container

└─ support-api app on port 8080

Tumhare actual project ka full flow

User browser / curl

↓

AWS ALB DNS

↓

Ingress: support-api-ingress

↓

Service: support-api:80

↓

Pod IPs: 10.0.12.238:8080, 10.0.11.27:8080

↓

Container: support-api

↓

Flask API endpoints:

/

/health

/version

Isko yaad rakhne ka easiest formula

ALB = Public gate

Ingress = Routing rule

Service = Internal stable address

Target Group = ALB backend list

Pod = Running app unit

Container = Actual app process

Screenshot me kya proof hai ki sab sahi chal raha hai?

Tumhare screenshot me:

Ingress address aa gaya

k8s-devapp-supporta-5aafe27a71-501100315.ap-south-1.elb.amazonaws.com

Matlab ALB create ho gaya ✓

Backend pods show ho rahe hain

support-api:80 (10.0.12.238:8080,10.0.11.27:8080)

``

Matlab service ke peeche pods registered hain ✓

Event successful hai

SuccessfullyReconciled

Matlab Load Balancer Controller ne Ingress ko successfully process kar diya ✓

API response aa raha hai

```
{"app":"support-api","message":"Support API is running on Amazon EKS","version":"v1"}
```

```
{"app":"support-api","status":"healthy","version":"v1"}
```

```
{"version":"v1"}
```

Matlab public ALB se app access ho rahi hai ✓

Interview-ready explanation

“I exposed my support-api microservice on EKS using AWS Load Balancer Controller and Kubernetes Ingress. The Ingress defined the HTTP routing rule, the controller provisioned an internet-facing ALB, and the ALB routed traffic to the Kubernetes Service. The Service forwarded traffic to healthy support-api pod IPs on container port 8080. I validated the setup by calling `/`, `/health`, and `/version` through the ALB DNS.”

Bhai practical advice

Tumhe abhi sab details ratne ki zarurat nahi hai. Bas daily 2-minute ye diagram revise karo:

User → ALB → Ingress → Service → Pod → Container → App